

Processamento de Linguagens

Trabalho Prático

Grupo 05

Cátia Eira (a107382)

Filipa Cosquete (a106837)

Marco Ferreira (a106857)



Índice

1. Introdução.....	2
2. Opções de Implementação.....	2
3. Análise Léxica.....	2
4. Análise Sintática.....	4
4.1. Estrutura Gramatical.....	4
4.2. Gestão de Conflitos (<i>Shift/Reduce</i>).....	5
4.3. Construção da AST.....	5
5. Análise Semântica.....	6
5.1. Tabela de Símbolos.....	6
5.2. Analisador Semântico.....	6
6. Tradução e Geração de Código.....	7
6.1. Declarações de variáveis e avaliação de expressões.....	8
6.2. Instruções de saída e entrada.....	8
6.3. Comandos de controlo de fluxo.....	8
6.3.1. IF.....	8
6.3.2. GOTO.....	9
6.3.3. Ciclos DO.....	9
6.3.4. Arrays.....	10
6.3.5. Funções.....	10
6.3.6. Subroutines.....	11
7. Conjunto de Testes.....	11
8. Conclusão.....	11

Repositório GitHub: <https://github.com/catiaeira/PL-2526-G05>

1. Introdução

Este relatório detalha o desenvolvimento de um compilador para a linguagem de programação Fortran 77, realizado no âmbito da unidade curricular de Processamento de Linguagens. O projeto foca-se na construção de um sistema capaz de processar código-fonte através de análise léxica, sintática, semântica e, finalmente, a tradução para o código de máquina de uma Máquina Virtual (VM) específica disponibilizada para o efeito. A implementação foi concretizada recorrendo à linguagem *Python* e às ferramentas *ply.lex* e *ply.yacc*.

2. Opções de Implementação

Decidimos ser importante manter-nos o mais fiéis possível ao Fortran 77, implementando um grande número de funcionalidades para além das referidas nos exemplos. Aquilo não implementado foi deixado de lado por questões de tempo, simplicidade ou pela inabilidade de ser convertido para a VM usada (ex.: gestão de I/O com ficheiros).

Decidimos usar o formato de colunas fixas, estando então as primeiras 5 colunas reservadas para *labels*, e a 6ª para indicações de continuar a linha anterior. O texto é pré-processado para truncar todas as linhas com mais de 72 caracteres (o limite aceito pelo Fortran¹).

Foi escolhido manter as diferentes partes do compilador o mais modular possível, de modo a fazer o *debug* mais simples e a divisão de responsabilidades mais óbvias. Assim, a *pipeline* do compilador realizado é a seguinte:

Lexer -> Parser -> Análise semântica -> Geração de código

Por questões de simplicidade, alguns detalhes não foram implementados:

- Fortran 77 é uma linguagem que não “olha” para os espaços entre as palavras, sendo “INTEGER A, B” tão válido quanto “INTEGERA,B”. Outro pormenor é o facto de não haver palavras reservadas, sendo válido em Fortran escrever “INTEGER INTEGER, B”.
- Há várias formas de declarar variáveis. Decidimos não implementar algumas delas, em particular aquelas que usam uma *keyword* (ASSIGN, DIMENSION) quando existem alternativas (ex.: INTEGER x, arr(10))

3. Análise Léxica

A primeira fase de criar o *lexer* foi o estudo inicial da linguagem e de quais palavras serviriam como *tokens*.²

Para aplicar o formato das colunas fixas são usados *states* exclusivos: o INITIAL (colunas 1-6) e o STMT (colunas 7-72).

¹ [Standard Fixed Format \(FORTRAN 77 Language Reference\)](#)

² [Keywords in Fortran Wiki](#)

No INITIAL:

- Caso encontre um 'C', 'c', '!' ou '*' na primeira coluna, ou um '!' nas colunas 7+, o resto da linha é descartada, visto tratar-se de um comentário.³
- Se encontrar algum símbolo na coluna 6, este é considerado uma linha de continuação, e a linha anterior não recebe um *token* de NEWLINE. Caso existam *labels* na mesma linha que um *continue*, estes são ignorados. Se a linha não for de continuação, é emitido um *token* NEWLINE e a posição no *lexer* retorna ao início da linha, para ler as *labels*.
- Se encontrar um número em alguma das colunas 1-5 este será designado como uma LABEL, e passa-se à análise do *statement*.⁴

No STMT:

- Quebras de linha (\n) iniciam o estado INITIAL.
- Caso receba uma palavra que comece com letras, esta é admitida um tipo com base nas *keywords* implementadas (ex.: CALL, IF, RETURN). Se a palavra não corresponder a nenhuma *keyword*, é designada como um ID.
- Com exceção dos tipos literais que veremos de seguida, o resto das implementações são expressões regulares triviais ou sem nenhum processamento especial.

As *keywords* aceites são:

CALL, CONTINUE, DO, ELSE, ELSEIF, END, ENDIF, FUNCTION, GOTO, IF,
PARAMETER, PRINT, PROGRAM, READ, RETURN, SAVE, STOP, SUBROUTINE, THEN,
WRITE, INTEGER, REAL, CHARACTER, COMPLEX, LOGICAL, DOUBLEPRECISION

Juntamente com os *tokens*:

ID, INT_LITERAL, REAL_LITERAL, DOUBLE_LITERAL, STRING_LITERAL, EQ, NE, LT,
LE, GT, GE, AND, OR, NOT, EQV, NEQV, TRUE, FALSE, POWER, CONCAT, NEWLINE,
LABEL

E os literais: (,), ', ' , :, +, -, *, /, =

Os tipos de dados lidos e os *regex* correspondentes são os seguintes:

- DOUBLE_LITERAL: (\d+\.\d*|\.\d+|\d+)[Dd][+-]?\d+
- REAL_LITERAL: (\d+\.\d*|\.\d+)([Ee][+-]?\d+)?|\d+[Ee][+-]?\d+
- INT_LITERAL: d+
- STRING_LITERAL: '([^\']|'')*'

³ [Comments and Blank Lines \(FORTRAN 77 Language Reference\)](#)

⁴ [Standard Fixed Format \(FORTRAN 77 Language Reference\)](#)

4. Análise Sintática

O analisador sintático foi desenvolvido utilizando a biblioteca *PLY*, mais especificamente o módulo *ply.yacc*, responsável pela construção automática de um parser LR a partir de regras gramaticais definidas em *Python*.

A análise sintática tem como objetivo validar a estrutura do código-fonte escrito numa variante simplificada de Fortran e, simultaneamente, construir uma representação intermédia do programa sob a forma de uma AST (*Abstract Syntax Tree*).

Cada regra da gramática foi implementada através de funções *Python* no formato exigido pelo *ply.yacc*. Durante as reduções efetuadas pelo *parser*, são criados nós da AST que representam semanticamente as construções da linguagem.

O parser suporta diversas estruturas da linguagem, incluindo:

- Programas principais (PROGRAM), funções e subrotinas
- Declarações de variáveis
- Expressões aritméticas e lógicas
- Estruturas condicionais (IF, ELSEIF, ELSE)
- Ciclos DO e instruções GOTO
- Chamadas de procedimentos (CALL)
- Operações de entrada e saída (READ, WRITE, PRINT)
- Instruções de controlo (RETURN, STOP, CONTINUE)

4.1. Estrutura Gramatical

A gramática foi organizada em múltiplos blocos funcionais, cada um responsável por uma componente específica da linguagem. Um ficheiro (FILE) é o “início” do código, que nada mais é que uma sequência de Top Level, onde:

```
TopLevel : MainProgram | SubroutineDef | FunctionDef
```

Isto permite que um ficheiro contenha programas principais, funções e subrotinas.

As declarações de variáveis suportam diferentes tipos de dados e incluem *arrays* uni e multidimensionais. A gramática foi construída de forma incremental, começando pelas declarações mais simples e adicionando progressivamente suporte para as funcionalidades mais avançadas.

As estruturas condicionais foram implementadas com suporte tanto para instruções simples de uma linha como para blocos completos IF-THEN-ELSE, com cadeias de ELSEIF possíveis.

Os ciclos DO foram modelados segundo o formato tradicional do Fortran, com limites inferiores, superiores e incremento opcional, juntamente com a LABEL associada. Ao contrário dos formatos IF, ciclos DO não têm o corpo embutido na sua declaração. Esta alteração deve-se à necessidade de permitir LABELS que não a final dentro do *loop* (ex.: continuações de linha).

Relativamente às expressões, foi criada uma hierarquia de regras sintáticas para representar corretamente a precedência dos operadores. Esta separação permitiu garantir que operações lógicas, relacionais e aritméticas são avaliadas pela ordem correta. A exponenciação recebeu tratamento especial devido à sua associatividade à direita.

Toda a estrutura gramatical foi desenhada com o objetivo de minimizar ambiguidades e facilitar a construção posterior da AST.

4.2. Gestão de Conflitos (*Shift/Reduce*)

Durante o desenvolvimento do parser surgiram potenciais conflitos sintáticos típicos de gramáticas LR, particularmente conflitos do tipo *shift/reduce*. Os casos mais relevantes:

- As expressões levaram à necessidade de criar uma hierarquia estrita com base na precedência e associatividade dos operadores (*reduce/reduce*);
- O *dangling else*, que ocorria quando o parser não conseguia determinar imediatamente a que instrução IF pertence um determinado ELSE (*shift/reduce*);
- A definição de um *array* e *function call* terem a mesma sintaxe (ID (OptExprList)) torna-as indistinguíveis para o *parser*, o que levou à junção de ambos para formar o *VarOrCall* (*reduce/reduce*);

Eliminar os conflitos sintáticos permitiu tornar a gramática mais explícita e legível sem usar *precedence tables*.

4.3 Construção da AST

Durante o processo de redução sintática, o parser constrói simultaneamente uma AST, responsável por representar o programa analisado.

Cada nó da árvore corresponde a uma construção da linguagem, como atribuições, operações aritméticas, estruturas condicionais ou chamadas de funções. É construída de forma hierárquica, através de dicionários aninhados:

Programas contêm blocos	>	Blocos contêm instruções	>
Instruções contêm expressões	>	Expressões contêm operadores e operandos.	

Cada dicionário tem um valor “*node*”, representativo da gramática que representa. Isto permite separar explicitamente os atributos, tal como as referências a variáveis, os acessos a *arrays*, e chamadas de função. Esta distinção melhora significativamente as fases posteriores de análise semântica e geração de código.

5. Análise Semântica

A fase de análise semântica é responsável por garantir que o programa respeita as regras e restrições da linguagem que não podem ser validadas apenas pela sintaxe. O analisador semântico valida o significado do código, garantindo a consistência de tipos, a existência de variáveis, o uso correto de rotinas.

5.1. Tabela de Símbolos

A base da análise assenta na classe *SymbolTable*, estruturada para gerir os recursos através de *scopes* isoladas:

- **Scope de Variáveis:** Uma pilha de dicionários. Permite o isolamento completo de variáveis entre o programa principal e as sub-rotinas/funções..
- **Scope de Labels:** Uma pilha de dicionários que rastreia os *labels*. Garante que um label usado num ciclo DO ou GOTO pertence exclusivamente ao *scope* local da rotina atual.
- **Dicionário Global de Rotinas:** Regista as assinaturas (nome, parâmetros e tipo de retorno) do programa, funções e sub-rotinas definidas pelo utilizador.
- **Dicionário da Biblioteca:** Armazena de forma estática as funções intrínsecas predefinidas (como MOD, SIN, ABS), permitindo validar a sua invocação.

As funções da tabela de símbolos tratam então de manter as informações sobre as diversas componentes organizadas, levantando erros quando algo não está de acordo com o pretendido.

5.2. Analisador Semântico

A análise em si utiliza uma estratégia de múltiplos passes sobre a AST para lidar de forma robusta com *forward references* e assinaturas dinâmicas:

- **Passe 1** (Registo de Rotinas): Percorre apenas o primeiro nível da árvore, e regista na tabela global a existência de todas as funções e subrotinas, marcando temporariamente os seus parâmetros com o tipo UNKNOWN.
- **Passe 2** (Análise de Subrotinas/Funções): Visita detalhadamente o corpo de cada função e subrotina. Este passo cria um novo *scope* local na tabela de símbolos, infere e resolve os tipos reais dos parâmetros (através de declarações explícitas ou tipagem implícita) e atualiza a assinatura global da rotina.
- **Passe 3** (Análise do Programa Principal): Visita e valida o bloco principal PROGRAM, onde o analisador recorre ativamente às operações de declaração, inicialização e procura fornecidas pela *SymbolTable*, de forma a validar as restrições semânticas impostas pela linguagem.

O analisador, tal como o *parser*, foi desenhado para ser tolerante a falhas (*error recovery*). Isto significa que, sempre que uma operação da tabela de símbolos falha ou uma restrição é violada, o analisador coleciona os objetos *SemanticError* levantados numa lista central, em vez de abortar imediatamente a execução. Isto permite expor múltiplos erros ao utilizador de uma só vez.

As principais validações dividem-se em cinco categorias:

- **Declaração e Inicialização:** Através das operações de declaração e inicialização, valida e impede definições duplicadas na mesma *scope*. Deteta o uso de variáveis antes de serem inicializadas, e, caso necessário, aplica as regras *default* de tipagem implícita (letras de I a N assumem o tipo INTEGER, as restantes tornam-se REAL).
- **Verificação de Tipos:** Utiliza as operações de procura para validar se os operandos em expressões aritméticas, lógicas ou relacionais são compatíveis. Garante a coerção permitida pela linguagem (promoção automática de INTEGER para REAL, mas nunca o oposto), e obriga a que condições de blocos IF / ELSEIF resultem estritamente em LOGICAL.
- **Validação de Índices de Arrays:** Intercepta os acessos a vetores/matrizes e obriga a que cada expressão utilizada como índice seja estritamente avaliada como sendo do tipo INTEGER.
- **Invocação de Rotinas:** Cruza dados com os métodos de validação da tabela. Garante que as chamadas a funções e subrotinas (incluindo as da biblioteca intrínseca como MOD) correspondem exatamente ao número e tipo de argumentos esperados, prevenindo ainda incompatibilidades de comportamento (como usar um comando CALL para invocar uma função).
- **Controlo de Labels:** Tendo sido acordado que a definição de uma *label* corresponde à sua existência nas colunas iniciais, e que a sua declaração corresponde à sua utilização num comando DO ou GOTO, efetua uma validação no fim da análise do programa, para garantir que todas as *labels* referenciadas por comandos GOTO ou DO foram efetivamente definidos no código. Impede também números de etiqueta duplicados.

6. Tradução e Geração de Código

Nesta etapa do projeto foi implementado o módulo responsável por transformar a representação intermédia produzida pelo parser numa sequência de instruções para a máquina virtual disponibilizada. Como a VM é baseada numa *stack*, a tradução teve de ser pensada de forma a preservar sempre a ordem de avaliação das expressões, o armazenamento de variáveis e o fluxo de execução do programa.

A geração de código foi organizada de forma incremental, começando pelos casos mais simples, validando-os, e avançando depois para as construções mais elaboradas da linguagem. Esta abordagem permitiu desenvolver cada funcionalidade de forma progressiva e facilitar a integração com as etapas anteriores do compilador.

Nota: a tabela de símbolos mencionada nesta secção não é a mesma usada na análise semântica, uma vez que a informação necessária em cada módulo do compilador não é a mesma.

6.1. Declarações de variáveis e avaliação de expressões

O primeiro passo realizado foi suportar declarações de variáveis. Para inicializar cada variável, é inserido o seu nome na tabela de símbolos, que devolve o índice global ao qual a variável ficou associada. Esta gestão dos índices (que no fundo se resume a incrementar um índice global de cada vez que é inserida uma variável) é feita internamente na tabela de símbolos, que pode ser inicializada com um valor inicial configurável para o índice global, o que é útil para as funções, conforme será explicado mais à frente. No que toca ao código da VM, para cada variável é inserido um 0 na *stack*, seguido de um *STOREG* com o índice devolvido pela tabela de símbolos.

Foi também necessário tratar a avaliação de expressões aritméticas, lógicas e relacionais. Como a VM opera em *stack*, a estratégia natural consiste em avaliar primeiro os operandos e só depois aplicar a instrução correspondente ao operador. Assim, expressões compostas são traduzidas de forma recursiva, respeitando a precedência definida pelo parser e garantindo que os resultados intermédios ficam sempre disponíveis no topo da pilha.

Esta fase inclui ainda a tradução de literais, referências a variáveis (feitas usando o índice guardado na tabela de símbolos com *PUSHG idx*) e operações unárias, como a negação lógica. A principal preocupação aqui foi assegurar que cada expressão produz exatamente um valor compatível com o tipo esperado pela instrução seguinte.

6.2. Instruções de saída e entrada

Depois de suportadas as expressões básicas, foram implementadas as instruções de escrita e leitura, fundamentais para testar qualquer programa e observar o seu comportamento.

No caso do *PRINT*, o código gerado distingue entre diferentes categorias de argumentos: literais, variáveis, expressões, e chamadas de funções. Para cada item, é gerado código segundo a sua natureza, escolhendo-se a instrução de escrita adequada ao tipo de dados em causa (e.g. *WRITES* para strings literais, *WRITEI* para *INTEGER* e *LOGICAL*, etc).

Já para o *READ*, foi necessário ter em conta o tipo da variável de destino para gerar a instrução correta de conversão. Por exemplo, se a variável de destino for um *INTEGER* ou *INTEGER_ARRAY*, as instruções geradas deverão ser *READ* e *atoi* (além das instruções para referenciar a variável, conforme descrito na avaliação de expressões).

6.3. Comandos de controlo de fluxo

Com a base da geração de código implementada, seguiu-se a tradução dos comandos de controlo de fluxo, nomeadamente *IF*, *GOTO* e ciclos *DO* com *labels*.

6.3.1. IF

O código da VM para um *IF* deverá ter uma forma semelhante a esta, considerando apenas um *IF* e um *ELSE*:

```

<argumento da condição>
JZ elseN
<código correspondente ao THEN>
JUMP endifN
elseN:
<código correspondente ao ELSE>
endifN:
<restante do código do programa>

```

Considerando isto, a tradução do IF deverá ser feita através de *labels* geradas automaticamente com um identificador único, que é um simples contador global. A condição é avaliada primeiro e, de seguida, o valor resultante determina o salto para a secção else ou para o final do comando. Esta solução é especialmente útil para suportar estruturas com múltiplos ramos, incluindo casos com ELSE IF e ELSE, pois permite organizar o código gerado como uma sequência linear de blocos ligados por saltos condicionais e incondicionais.

6.3.2. GOTO

O GOTO é a forma mais direta de controlo de fluxo e traduz-se num salto incondicional para a *label* indicada no programa fonte. As *labels*, quando detectadas ao processar uma linha, são diretamente colocadas no código gerado, sendo o GOTO executado com um simples JUMP para a *label* indicada.

6.3.3. Ciclos DO

A tradução de ciclos DO exigiu mais cuidado do que os restantes comandos, porque o Fortran 77 usa *labels* para identificar o fim do ciclo. A abordagem seguida consiste em gerar uma *label* para o início do ciclo, testar a condição de continuação e, no final de cada iteração, atualizar a variável de controlo e voltar ao início, seguindo uma estrutura semelhante a esta:

```

LABEL:
PUSHG <variável do ciclo>
PUSHG <stop>
INFEQ
JZ LABELend
<código do ciclo>
JUMP 10
LABELend:
<resto do programa>

```

Como os limites e o passo do ciclo podem ser expressões ou variáveis, foi necessário assegurar que esses valores ficam disponíveis durante toda a duração do ciclo. Quando isso não acontece diretamente, são criadas variáveis temporárias para preservar o valor de fim e o incremento. No final do ciclo, essas variáveis temporárias são removidas, evitando a acumulação de espaço desnecessário na stack.

A correspondência entre o DO e o CONTINUE associado é, nesta fase, resolvida através da *label* do ciclo. Isto permite fechar corretamente o bloco do ciclo mesmo na presença de instruções intermédias arbitrárias.

6.3.4. Arrays

Passando agora às funcionalidades mais avançadas, aqui surgiu um problema importante: o *parser* não distingue de forma nativa uma referência a *array* de uma chamada de função, já que ambas aparecem com uma sintaxe semelhante, isto é, um identificador seguido de argumentos entre parênteses. Por esse motivo, a geração de código deve ser capaz de resolver esta ambiguidade.

A solução adotada foi consultar a tabela de símbolos. Se o identificador existir como variável do programa, então a estrutura é interpretada como acesso a *array*; caso contrário, é tratada como chamada de função. Este mecanismo permite desambiguar corretamente casos que, do ponto de vista sintático, são idênticos.

Na tradução de acesso a *arrays*, foi também necessário adaptar o índice do Fortran, que começa em 1, ao índice da VM, que trabalha em base 0. Essa conversão é feita automaticamente durante a geração do código, subtraindo uma unidade ao índice calculado. Assim, a semântica original da linguagem é preservada sem alterar o comportamento do programa fonte.

6.3.5. Funções

As funções representam a parte mais complexa da geração de código implementada, tanto pela necessidade de gerir parâmetros e valores de retorno, como pela interação direta com a *stack* da VM.

A nível de execução, a chamada de uma função exige várias etapas. Primeiro, é reservado espaço para o valor de retorno, para que a função tenha um local próprio onde possa escrever o resultado. De seguida, são colocados na *stack* os argumentos, na ordem esperada. Depois disso, é efetuado o salto para o corpo da função.

No final da execução da função, é necessário remover da *stack* as variáveis locais antes de devolver o controlo ao *caller*. Esta limpeza é importante para manter a integridade da pilha e evitar que dados temporários interfiram com chamadas futuras. Após o RETURN, o código do chamador também remove os argumentos que foram empilhados antes da chamada, fechando corretamente o ciclo de passagem de parâmetros. Desta forma, fica no topo da *stack* o valor de retorno gerado pela função.

Outro detalhe relevante é o tratamento dos tipos implícitos. Em Fortran 77, quando um identificador não é declarado explicitamente, o seu tipo pode ser inferido pela regra padrão da linguagem. Esse comportamento foi respeitado na geração de código, particularmente para parâmetros de funções, de modo a manter consistência com o restante compilador.

Também foi feita uma distinção entre funções intrínsecas e funções definidas pelo utilizador. Funções intrínsecas, como MOD, são traduzidas diretamente para instruções da VM, ou para sequências pequenas de instruções equivalentes. Já as funções definidas pelo

utilizador exigem uma chamada completa, com gestão explícita da *stack* e do valor de retorno, tal como já foi descrito.

Por fim, a integração das funções com o corpo principal do programa exige a utilização de algumas *labels* auxiliares para manter o fluxo de execução esperado. Mais concretamente, é necessário colocar uma “*endprogram:*” no final de todo o código e, no final do corpo principal do programa, um “*JUMP endprogram*”. Isto tem que ser feito porque o código das funções deve ser gerado após o corpo principal, e sem o salto para o final, todas as funções seriam executadas uma vez no final da execução do programa.

6.3.6. Subroutines

As SUBROUTINE não foram implementadas nesta versão, mas, do ponto de vista da geração de código, seriam apenas um caso mais simples do mecanismo das funções. A principal diferença seria a ausência de valor de retorno, o que simplificaria a gestão inicial da *stack*. Mesmo assim, manteriam a necessidade de empilhar argumentos, executar o corpo da rotina e remover os parâmetros no fim da chamada.

7. Conjunto de Testes

Por fim, foram criados alguns exemplos de programas Fortran 77, de modo a testar a correção do compilador. Existem 10 programas válidos (dos quais 5 são os apresentados no enunciado), e 11 inválidos.

Cada ficheiro de teste é submetido a todo o processo de análise sintática, semântica, e geração de código. Numa fase inicial do programa de testes, ele apenas verificava se o código passava ou falhava, não se importando com o onde, nem o como. Escolhemos então alterá-lo de forma a validar os erros obtidos. Apesar da fase de definição de um teste requerer que tanto os erros sintáticos como semânticos sejam referidos manualmente, isto poupa trabalho na análise dos resultados.

Dos 21 testes definidos, apenas 2 falham, devido à falta de implementação de algumas funcionalidades na geração de código máquina, nomeadamente do tipo CHARACTER, e dos *arrays* multidimensionais. No entanto, estes testes permitem ainda assim solidificar a confiança na análise sintática e semântica.

8. Conclusão

Houveram várias dificuldades na criação deste projeto, várias delas com base nos poucos exemplos de código de Fortran 77 disponíveis *online*, tornando difícil o estudo da linguagem e a garantia que o compilador se comporta como esperado.

No entanto, o compilador obtido cumpre todos os requisitos mínimos: consegue processar os exemplos dados no enunciado, é modular nas suas partes, tem comentários a explicar decisões no código, e é o mais fiel possível à linguagem.

Para trabalho futuro, escolheríamos aplicar otimizações no *parser* e na tradução do código, e expandir as funcionalidades suportadas.